SubTask-3

Tilak Asodariya

1 Introduction

The objective of this task was to implement and evaluate a GPT-style decoder-only Transformer for poetry generation. The implementation was developed from scratch using PyTorch primitives without relying on high-level Transformer APIs. The performance of the Transformer was compared against Bigram and LSTM language model baselines.

2 Dataset and Preprocessing

The Hugging Face poetry dataset (`merve/poetry`) was used.
The dataset was shuffled and split into:

- Training Set: 80%
- Validation Set: 10%
- Test Set: 10%

Only the `content` column was used.
Preprocessing steps:

- Removed quotation marks
- Removed brackets and parentheses
- Removed punctuation attached to words
- Preserved capitalization
- Applied word-level tokenization

2.1 Dataset Statistics

Metric	Value
Vocabulary Size	11,167
Training Tokens	73,733
Validation Tokens	14,205
Test Tokens	11,214

Table 1: Dataset Statistics

3 Bigram Language Model

A Bigram Language Model was implemented using transition counts stored in Python dictionaries.

The model predicts:

$$P(w_{i+1}|w_i)$$

Generation was performed through weighted random sampling.

3.1 Results

Metric	Value
Validation Loss	16.1933
Token Accuracy	6.49%
Unseen Validation Transitions	64%

Table 2: Bigram Results

4 LSTM Baseline

Sequences were generated using a sliding window of length 20.

$$X = [w_i, \ldots, w_{i+19}]$$

$$Y = [w_{i+1}, \ldots, w_{i+20}]$$

4.1 Architecture

- Embedding Layer
- LSTM Layer
- Fully Connected Output Layer

Final Hyperparameters:

- Embedding Dimension = 64
- Hidden Dimension = 128
- Number of Layers = 2
- Dropout = 0.3

4.2 Results

Metric	Value
Training Loss	2.4774
Validation Loss	12.0942
Validation Accuracy	8.83%

Table 3: LSTM Results

5 Decoder-Only Transformer

A GPT-style decoder-only Transformer was implemented from scratch.
Components implemented:

- Token Embeddings
- Positional Embeddings
- Causal Self-Attention
- Multi-Head Attention
- Masked Attention Mechanism
- Feed Forward Network
- Decoder Blocks
- Autoregressive Generation

5.1 Initial Configuration

Hyperparameter	Value
Batch Size	64
Sequence Length	20
Embedding Dimension	128
Attention Heads	4
Decoder Layers	2
Feed Forward Dimension	512
Dropout	0.3
Epochs	10
Learning Rate	10^{-3}

Table 4: Initial Transformer Hyperparameters

Approximate model size:

3.26 Million Parameters

5.2 Initial Results

Metric	Value
Training Loss	1.7403
Validation Loss	13.3284
Validation Accuracy	11.85%

Table 5: Initial Transformer Results

Although the Transformer achieved the highest token accuracy, substantial overfitting was observed.

6 Transformer Hyperparameter Tuning

To reduce overfitting, the architecture was modified.

Hyperparameter	Value
Batch Size	32
Sequence Length	50
Embedding Dimension	64
Attention Heads	4
Decoder Layers	2
Feed Forward Dimension	256
Dropout	0.3
Epochs	10
Learning Rate	3×10^{-4}
Weight Decay	10^{-4}

Table 6: Tuned Transformer Hyperparameters

Approximate model size:

1.65 Million Parameters

6.1 Tuned Results

Metric	Value
Training Loss	1.8658
Validation Loss	10.7994
Validation Accuracy	12.69%

Table 7: Tuned Transformer Results

The tuned model achieved significantly better validation performance while reducing overfitting.

7 Inference and Decoding Experiments

7.1 Greedy Decoding

Greedy decoding always selects the highest-probability token.

Observations:

- High coherence
- High repetition
- Limited creativity

7.2 Temperature Sampling

Temperature controls randomness during generation.

Temperature	Creativity	Repetition	Coherence
0.5	Low	High	High
0.7	Low	High	High
1.0	Medium	Medium	Good
1.5	High	Low	Moderate

Table 8: Temperature Sampling Analysis

7.3 Top-k Sampling

k	Creativity	Repetition	Coherence
5	Low	High	High
20	Medium	Medium	Good
50	High	Low	Moderate

Table 9: Top-k Sampling Analysis

7.4 Top-p Sampling

p	Creativity	Repetition	Coherence
0.8	Low	High	High
0.9	High	Low	Good
0.95	Very High	Low	Lower

Table 10: Top-p Sampling Analysis

Top-p sampling with $p = 0.9$ produced the best balance between creativity and coherence.

8 Prompt Conditioning

Prompt conditioning was evaluated using:

- Love is
- The moon
- In the night
- I remember

The generated poems adapted their vocabulary and imagery according to the prompt. Prompts related to love generated emotional and romantic themes, while prompts related to the moon and night produced imagery associated with nature and landscapes. This demonstrates successful conditioning on the initial prompt.

9 Overall Comparison

Model	Validation Loss	Accuracy
Bigram	16.1933	6.49%
LSTM	12.0942	8.83%
Transformer (Initial)	13.3284	11.85%
Transformer (Tuned)	10.7994	12.69%

Table 11: Model Comparison

10 Conclusion

A decoder-only Transformer for poetry generation was successfully implemented from scratch. The model incorporated causal self-attention, positional embeddings, masking mechanisms, decoder blocks, and autoregressive generation. Compared to the Bigram and LSTM baselines, the Transformer achieved the highest token prediction accuracy and generated richer poetic language. Hyperparameter tuning reduced overfitting and significantly improved validation performance. Among the decoding strategies evaluated, Top-p sampling with $p = 0.9$ produced the best overall generation quality.